31. ACLs and umask

31. ACLs and umask

Phase: Phase 2 — Permissions and identity
Estimated time: 1 day
Slug: `31-permissions-acl`

Learning objectives

- Explain umask defaults
- Read basic ACLs with getfacl
- Decide when ACLs are warranted
- Avoid ACL sprawl

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Classic Unix permissions (owner, group, other) cannot express "alice may read, bob may write, everyone else nothing" on one file. **ACLs** (access control lists) can. And every file your programs create gets its initial permissions from the **umask**, so a wrong umask silently leaks secrets or breaks shared directories. This module ties both together so you can predict who can access what before you test it.

Core concepts

Concept	Meaning
umask	Bits removed from the default creation mode
default mode	666 for files, 777 for directories (before umask)
ACL entry	Extra rule such as `user:alice:rw-` or `group:dev:r-x`
mask	Upper bound for named users, named groups, and the owning group
default ACL	ACL on a directory inherited by new files inside it
setfacl/getfacl	Tools to set and show ACLs

umask arithmetic (subtract the bits, do not subtract the number blindly):

```
file default   666  = rw- rw- rw-
umask 022          = --- -w- -w-   (bits to remove)
result         644  = rw- r-- r--

dir default    777  = rwx rwx rwx
umask 022
result         755  = rwx r-x r-x

umask 077 -> files 600, dirs 700   (private)
umask 002 -> files 664, dirs 775   (group collaboration)
```

Files never get the execute bit from creation alone, which is why `touch` with umask 022 produces 644, not 755.

ACL evaluation order:

```
1. owner matches        -> owner entry
2. named user matches   -> user:NAME entry  AND mask
3. group(s) match       -> owning group or named group entries AND mask
4. otherwise            -> other entry
```

Worked examples

Check and change umask in the current shell:

```
umask            # prints 0022
umask -S         # u=rwx,g=rx,o=rx
umask 077
touch private.txt && mkdir private.d
ls -ld private.txt private.d
```

Expected:

```
-rw------- 1 you you 0 Oct 21 10:30 private.txt
drwx------ 2 you you 4096 Oct 21 10:30 private.d
```

Return to the usual default and confirm file mode 644:

```
umask 022
touch normal.txt
stat -c '%a %n' normal.txt     # 644 normal.txt
```

Grant one extra user access with an ACL:

```
sudo apt install acl            # or: sudo dnf install acl
setfacl -m u:bob:rw report.txt
getfacl report.txt
ls -l report.txt                # note the trailing +
```

Sample getfacl output:

```
# file: report.txt
# owner: you
# group: you
```

```
user::rw-
user:bob:rw-
group::r--
mask::rw-
other::r--
```

Remove one entry or all entries:

```
setfacl -x u:bob report.txt
setfacl -b report.txt
```

Shared project directory with inheritance:

```
sudo mkdir /srv/proj
sudo chgrp dev /srv/proj
sudo chmod 2775 /srv/proj                 # setgid keeps group dev
sudo setfacl -d -m g:dev:rwx /srv/proj    # default ACL for new files
sudo setfacl -d -m o::--- /srv/proj       # others get nothing
```

Persist umask for services: set `UMask=0027` in the systemd unit, or edit `/etc/login.defs` and shell profile for logins.

Mini-lab

1. Run `umask`. Predict the mode of a new file and a new directory. Create both and compare with `stat -c '%a %n'`.
2. Set `umask 027`, create again, predict, verify. Then restore your umask.
3. Create `report.txt`, give a second user (or `nobody` via sudo test) read access via `setfacl`. Confirm with `getfacl` and `sudo -u nobody cat`.
4. Lower the mask with `setfacl -m m::r-- report.txt` and observe that the named user's effective permission drops (getfacl shows `#effective:`).
5. Create the shared directory with a default ACL and verify a new file inside inherits the entries.

Decision guide / failure modes

Need	Use
Default privacy for new files	umask 077 or 027
Team shares a directory	group + setgid + umask 002, or default ACL
One extra user on one file	`setfacl -m u:NAME:perm`
Many users, many rules	groups first; ACLs only where groups do not fit
Backup/copy must keep ACLs	`cp -a`, `rsync -A`, `tar --acls`

Failure modes:

- **Subtracting numbers**: 666 - 022 happens to work, but 666 - 027 style math breaks with odd digits. Think in bits.
- **Forgetting the mask**: chmod on a file with ACLs changes the mask and can silently cut named-user access.
- **No default ACL on the directory**: new files do not inherit the access ACL; only default entries inherit.
- **Filesystem without ACL support**: mount option `acl` missing on some old or network filesystems.
- **`ls -l` hides detail**: a trailing `+` is the only hint; always run `getfacl` when permissions look odd.
- **Umask in cron/systemd differs** from your login shell, so scripts create files with different modes.

Common mistakes

- umask 000 creating world-writable files
- ACLs on every directory
- Forgetting defaults on shared project dirs

Exercises

Work in `exercises/31-permissions-acl/`. Read that folder's `README.md` fully before coding.

```
cd exercises/31-permissions-acl
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `file_mode_after_umask022` in `exercise.sh` so it prints the octal permission of a newly created regular file under umask 022. Derive it from the rule in the lesson (666 with the umask bits removed), then confirm by running `umask 022; touch t; stat -c %a t` in a scratch directory. Print only the three digits. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/31-permissions-acl/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does umask 022 yield for files?
2. When use ACL vs groups?
3. getfacl purpose?

Further reading

- https://man7.org/linux/man-pages/man1/umask.1p.html